

# SAVCNG ExcelDNA - Documentación Técnica Completa

---

**Versión:** 2.2.2.0

**Última Actualización:** 12 de junio de 2026

**Estado:** En Desarrollo Continuo

## Tabla de Contenidos

1. [Descripción General](#)
  2. [Arquitectura del Proyecto](#)
  3. [Componentes Principales](#)
  4. [Sistema de Validaciones Detallado](#)
    - [4.1 Validación de Decimales](#)
    - [4.2 Validación de Catálogos](#)
    - [4.3 Validación NS](#)
    - [4.4 Validación de Formato Texto](#)
    - [4.5 Validación de Bloqueo Dinámico](#)
  5. [FrmValidaciones.cs - Análisis Profundo](#)
  6. [Flujo de Trabajo](#)
  7. [Especificaciones Técnicas](#)
  8. [Guía de Desarrollo](#)
  9. [Patrones de Código y Mejores Prácticas](#)
  10. [Troubleshooting y Casos Especiales](#)
- 

## Descripción General

**SAVCNG ExcelDNA** es un complemento (Add-in) de Microsoft Excel desarrollado en C# (.NET Framework 4.8) que proporciona un sistema avanzado de validación de datos para archivos de censo de gobierno.

El proyecto utiliza **ExcelDNA** como framework de integración con Excel, permitiendo crear un menú personalizado en la cinta de opciones (Ribbon) de Excel desde el cual se pueden cargar archivos de censo y aplicar múltiples validaciones de integridad de datos.

### Propósito Principal

- Garantizar la calidad e integridad de los datos en archivos de censo
- Proporcionar una interfaz gráfica intuitiva para validación de datos
- Detectar y resaltar errores de formato y contenido
- Facilitar listas desplegadas (Data Validation) personalizadas
- Validar presencia de respuestas "No Aplica" (NS) con alertas contextuales
- Validar formato de texto: mayúsculas, sin espacios extra, caracteres válidos
- Crear reglas de bloqueo dinámico condicionadas a valores de otras celdas
- Soporte automático para matrices paralelas y evaluaciones fila por fila
- Mensajes de alerta personalizados que aparecen dinámicamente

### Tecnologías Utilizadas

- **Lenguaje:** C# 7.3
- **Framework:** .NET Framework 4.8
- **Librería de Interop:** Microsoft.Office.Interop.Excel
- **Plugin Framework:** ExcelDNA
- **UI:** Windows Forms (.NET Framework)

---

## Arquitectura del Proyecto

```
SAVCNG_ExcelDNA/  
├─ MenuCenso.cs           (Punto de entrada y menú)  
├─ FrmValidaciones.cs     (Lógica principal de validaciones)  
├─ FrmValidaciones.Designer.cs (Diseño de formulario)  
├─ Properties/  
│   └─ AssemblyInfo.cs    (Metadatos del ensamblado)  
├─ bin/                   (Ejecutables compilados)  
├─ obj/                   (Artefactos compilados)  
└─ documentacion/  
    └─ documentacion_savcng_YYYY-MM-DD.md (Documentación técnica)
```

### Patrón Arquitectónico

- **Model:** Las colecciones (List, Dictionary) almacenan datos de validación y rangos
- **View:** Windows Forms (FrmValidaciones) para la interfaz de usuario
- **Controller:** Métodos de eventos que procesan interacciones del usuario
- **Service:** Métodos estáticos y funciones auxiliares para lógica compartida

---

## Componentes Principales

### 1. MenuCenso.cs - Punto de Entrada

Responsable de la integración con Excel y la interfaz de usuario del ribbon.

#### Características Principales:

```
public class MenuCenso : ExcelRibbon  
{  
    // Define un menú personalizado en la cinta de Excel  
    public override string GetCustomUI(string RibbonID)  
}
```

#### Funcionalidades:

- **XML Ribbon:** Define la estructura del menú en la pestaña "SAVCNG - Censo"
- **Diálogo de Archivo:** Permite seleccionar archivos de censo (.xlsx, .xlsm, .xlsb)
- **Gestión de Workbook:** Abre el archivo y lo pasa a la ventana de validaciones

- **Manejo de Errores:** Captura excepciones durante la carga de archivos

## Flujo de Ejecución:

1. Usuario hace clic en "Cargar Censo" en el menú de Excel
2. Se abre un diálogo `OpenFileDialog`
3. El archivo seleccionado se abre en Excel mediante `excelApp.Workbooks.Open()`
4. Se instancia `FrmValidaciones` pasando el workbook
5. La ventana se muestra flotante sobre Excel

---

## 2. `FrmValidaciones.cs` - Motor de Validaciones

Este es el corazón del aplicativo. Implementa todas las validaciones de datos y maneja la interacción del usuario.

### 2.1 Variables de Estado Principales

```
private Excel.Workbook _libroCenso;
```

- **Propósito:** Referencia al archivo de Excel cargado
- **Tipo:** `Excel.Workbook` (objeto COM de Office)
- **Inicialización:** En el constructor, recibida desde `MenuCenso.cs`
- **Ciclo de vida:** Se mantiene mientras la ventana esté abierta
- **Notas:** Es un objeto COM, requiere manejo cuidadoso de memoria

```
private Excel.Range _rangoCapturado;
```

- **Propósito:** Almacena el rango de celdas seleccionado por el usuario
- **Tipo:** `Excel.Range` (objeto COM de Office)
- **Inicialización:** Se asigna en `btnCapturarRango_Click()`
- **Validación:** Se verifica que sea un rango (no imagen, gráfico, etc.)
- **Uso:** Base para aplicar todas las validaciones
- **Notas:** Puede contener múltiples áreas (rangos no contiguos)

---

## Sistema de Validaciones Detallado

### 4.1 Validación de Decimales

**Objetivo:** Detectar y resaltar celdas que contienen números con decimales cuando solo se permiten números enteros.

**Tipo:** Formato Condicional (Visual)

**Restricción:** No (solo resalta)

**Motor:** `ESNUMERO()` + `TRUNCAR()`

**Características:**

- Fondo amarillo con texto rojo
  - Detecta automáticamente celdas con decimales
  - Compatible con rangos de cualquier tamaño
- 

## 4.2 Validación de Catálogos

**Objetivo:** Crear listas desplegables (Data Validation) con opciones predefinidas.

**Tipo:** Data Validation - Lista

**Restricción:** Sí (solo permite valores de la lista)

**Motor:** Rango de opciones o entrada manual

**Características Avanzadas:**

- Soporte para celda única, rango de celdas y celdas combinadas
- Opción de extraer SOLO NÚMEROS del contenido
- Opción de mantener TEXTO EXACTO
- Opción de usar rango como referencia directa
- Limpieza automática de caracteres innecesarios
- Entrada manual de opciones separadas por comas

**Flujo de Configuración:**

1. Usuario selecciona rango de captura
  2. Sistema pregunta: ¿Entrada manual o seleccionar celdas?
  3. Si es manual: Ingresar opciones (ej: "1,2,9")
  4. Si es celda única: ¿Extraer números? ¿Mantener texto? ¿Usar como referencia?
  5. Si es rango múltiple: Igual análisis que celda única
  6. Aplicar validación con dropdown
- 

## 4.3 Validación NS (No Aplica)

**Objetivo:** Permitir números  $\geq 0$  o valor "NS", con alerta automática cuando se usa NS.

**Tipo:** Data Validation - Custom + Formato + Fórmula

**Restricción:** Sí (solo permite números  $\geq 0$  o "NS")

**Motor:** `=O(Y(ESNUMERO, >=0), "NS")`

**Contexto:** En censos, "NS" significa "No Sabe" o "No Aplica" y requiere justificación.

**Características:**

- Validación restrictiva con mensaje personalizado
- Alerta automática en fila siguiente cuando hay registros NS
- Mensaje combinado y formateado (Arial 9pt, Negrita, Color Dorado)
- Manejo de rangos no contiguos (Areas)
- Fórmula con CONTAR.SI para cada área

**Fórmula de Validación:**

```
=O(Y(ESNUMERO(A1);A1>=0);A1="NS")
```

**Fórmula de Alerta:**

```
=SI(SUMA(CONTAR.SI(A:B;"NS"))<>0;"Alerta: debe justificar NS";"
```

## 4.4 Validación de Formato Texto

**Objetivo:** Garantizar que el texto cumpla formato específico: mayúsculas, sin espacios extra.

**Tipo:** Data Validation - Custom

**Restricción:** Sí (previene entrada inválida)

**Motor:** IGUAL() + ESPACIOS()

**Reglas Implementadas:**

1. ✓ Solo mayúsculas (A-Z, Ñ)
2. ✓ Espacios simples (sin dobles espacios)
3. ✓ Sin espacios al inicio o final

**Características:**

- Data Validation restrictiva (el usuario NO puede violar las reglas)
- Mensaje de error personalizado y detallado
- Compatible con celdas vacías (IgnoreBlank = true)
- Funciona con rangos de cualquier tamaño
- Valida en tiempo real mientras el usuario escribe

**Fórmula:**

```
=Y(IGUAL(A1;MAYUSC(A1));LARGO(A1)=LARGO(ESPACIOS(A1)))
```

**Mensaje de Error:**

El texto debe cumplir las siguientes reglas:

- Todo en MAYÚSCULAS.
- Sin dobles espacios.
- Sin espacios al inicio o al final.

## 4.5 Validación de Bloqueo Dinámico ★ (v2.2+)

**Objetivo:** Crear reglas de bloqueo condicional basadas en valores de otras celdas o rangos.

**Tipo:** Data Validation - Custom + Formatos Condicionales

**Restricción:** Sí (bloquea entrada si condición no se cumple)

**Motor:** **CONTAR.SI()** con detección inteligente de matrices

### Características Principales:

- Detección automática de matrices paralelas
- Direcciones dinámicas (Local para fila-por-fila, Global para búsqueda)
- Tres visuales dinámicos: Bloqueado (gris), Desbloqueado (normal), Obligatorio (rojo)
- Resalte de instrucción en amarillo (opcional)
- Mensaje de alerta personalizado (opcional, v2.2.2)
- Soporte para 6 operadores: =, <>, >, <, >=, <=

### Contexto de Uso:

Ejemplo: Campo "Especifique" que solo se activa si P1 = "Otra"

- └ Si P1 = "Otra" → Se desbloquea "Especifique"
- └ Si P1 ≠ "Otra" → Se bloquea (gris)
- └ Si se desbloquea pero vacío → Se resalta en rojo (obligatorio)

### Ejemplo Matriz Paralela:

Pregunta: Indique grado de educación por inciso

|            | Inciso A                 | Inciso B                 | Inciso C                 |
|------------|--------------------------|--------------------------|--------------------------|
| Primaria   | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> |
| Secundaria | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> |
| Terciaria  | <input type="checkbox"/> | <input type="checkbox"/> | <input type="checkbox"/> |

Especifique nivel (se desbloquea FILA POR FILA si selecciona Terciaria)

### Flujo de Configuración (6 Pasos):

- Paso 1:** Seleccionar rango/celda de condición
  - Una celda → Automáticamente fila por fila
  - Rango igual tamaño → Sistema pregunta "¿Fila por fila?"
  - Rango diferente tamaño → Automáticamente búsqueda global
- Paso 2:** Elegir operador (=, <>, >, <, >=, <=)
- Paso 3:** Ingresar valor de criterio (ej: "Otra", 6, "Sí")
- Paso 4:** ¿Resalte rojo? (cuando desbloqueado + vacío)
  - Sí → Rojo suave para indicar obligatorio
  - No → Solo gris cuando bloqueado

### 5. **Paso 5:** ¿Resalte de instrucción? (amarillo)

- Sí → Seleccionar celda con instrucción
- No → Continuar

### 6. **Paso 6:** ¿Mensaje de alerta personalizado? (v2.2.2 ★ NUEVO)

- Sí → Ingresar texto → Seleccionar ubicación
- No → Finalizar
- Aparece automáticamente cuando se cumple condición + campo vacío

### Implementación Técnica Clave:

```
// Detección automática de matriz
bool esFilaPorFila = false;

if (rangoCondicion.Count == 1)
{
    esFilaPorFila = true; // Una celda
}
else if (_rangoCapturado.Rows.Count == rangoCondicion.Rows.Count)
{
    // Misma altura - Preguntar usuario
    DialogResult respuesta = MessageBox.Show("¿Fila por Fila?", ...);
    esFilaPorFila = (respuesta == DialogResult.Yes);
}

// Direcciones dinámicas
string dirCondicionLocal = esFilaPorFila
    ? rangoCondicion.Cells[1, 1].Address.Replace("$", "") // A1 (relativo)
    : rangoCondicion.Address; // $A$1:$A$21 (fijo)

string dirCondicionGlobal = rangoCondicion.Address; // Siempre fijo para instrucción

// Fórmula de validación
string formulaValidacion = $"=CONTAR.SI({dirCondicionLocal};{criterioContarSi})>0";

// Formato gris (bloqueado)
string formulaSombreado = $"=CONTAR.SI({dirCondicionLocal};{criterioContarSi})=0";

// Formato rojo (desbloqueado + vacío, opcional)
string formulaRojo = $"=Y(CONTAR.SI({dirCondicionLocal};{criterioContarSi})>0;ESBLANCO({dirCapturada}))";

// Formato amarillo (instrucción, opcional)
string formulaInstruccion = $"=CONTAR.SI({dirCondicionGlobal};{criterioContarSi})>0";

// Mensaje dinámico (nuevo v2.2.2)
```

```
string formulaAlerta = $"=IF(COUNTIF({dirCondicionGlobal};
{criterioContarSi})>COUNTA({dirCapturadaGlobal});\"{textoAlerta}\",\"{\")\"";
```

### Estados Visuales Completos:

| Estado | Bloqueado | Visual          | Interacción       | Significado                        |
|--------|-----------|-----------------|-------------------|------------------------------------|
| 1      | Sí        | Gris CrissCross | NO puede escribir | No se cumple condición             |
| 2      | No        | Normal          | Puede escribir    | Se cumple y con valor              |
| 3      | Parcial   | Rojo suave      | Puede escribir    | Se cumple pero vacío (obligatorio) |
| 4      | N/A       | Amarillo        | Solo lectura      | Instrucción relacionada activa     |
| 5      | N/A       | Rojo en alerta  | Solo lectura      | Mensaje personalizado (v2.2.2)     |

## FrmValidaciones.cs - Análisis Profundo

### 5.1 Método: btnCapturarRango\_Click

**Responsabilidad:** Capturar el rango seleccionado por el usuario y detectar contexto.

#### Lógica:

1. Obtiene aplicación de Excel via ExcelDNA
2. Lee la selección actual del usuario
3. Valida que sea un rango (no imagen/gráfico)
4. Almacena en `_rangoCapturado`
5. Detecta número de pregunta automáticamente
6. Muestra feedback visual

#### Código:

```
if (seleccion is Excel.Range)
{
    _rangoCapturado = (Excel.Range)seleccion;

    string pregunta = ObtenerNumeroPregunta(_rangoCapturado);
    lblPregunta.Text = "Pregunta detectada: " + pregunta;
    lblRangoSeleccionado.Text = "Rango seleccionado: " +
    _rangoCapturado.Address.Replace("$", "");

    MessageBox.Show("Se capturó correctamente el rango: " +
    _rangoCapturado.Address,
        "Captura exitosa", MessageBoxButtons.OK, MessageBoxIcon.Information);
}
```

### 5.2 Método: btnAplicar\_Click - Estructura General



**Responsabilidad:** Orquestrar la aplicación de validaciones según checkboxes marcados.

### Estructura de Rutas:

```
btnAplicar_Click()
├─ Validación previa: _rangoCapturado != null
├─ if (chkDecimales.Checked)
│   ├─ Limpiar formatos previos
│   ├─ Crear fórmula: Y(ESNUMERO, TRUNCAR<>)
│   ├─ Aplicar FormatCondition (Amarillo/Rojo)
│   └─ Mostrar éxito
├─ else if (chkCatalogos.Checked)
│   ├─ Preguntar: ¿Manual o seleccionar?
│   ├─ Si manual: Ingresar opciones
│   ├─ Si seleccionar: ¿Números? ¿Texto? ¿Referencia?
│   ├─ Aplicar Validation xlValidateList
│   └─ Mostrar éxito
├─ else if (chkNS.Checked)
│   ├─ Crear fórmula: O(Y(ESNUMERO, >=0), "NS")
│   ├─ Aplicar Data Validation
│   ├─ Insertar alerta en fila siguiente
│   ├─ Calcular fórmula: SI(SUMA(CONTAR.SI...))
│   └─ Mostrar éxito
├─ else if (chkFormatoTexto.Checked)
│   ├─ Crear fórmula: Y(IGUAL, ESPACIOS)
│   ├─ Aplicar Data Validation
│   ├─ Configurar mensaje: "Mayúsculas, sin espacios..."
│   └─ Mostrar éxito
├─ else if (chkBloqueo.Checked) ★ COMPLEJO (v2.2.2)
│   ├─ Paso 1: Seleccionar condición
│   │   ├─ Detectar: 1 celda, igual tamaño, diferente tamaño
│   │   └─ Decidir: esFilaPorFila = true/false
│   ├─ Paso 2: Seleccionar operador
│   ├─ Paso 3: Ingresar valor
│   ├─ Paso 4: ¿Resalte rojo? (Sí/No)
│   ├─ Aplicación de reglas:
│   │   ├─ Data Validation (CONTAR.SI fila-por-fila)
│   │   ├─ Formato Gris (CrissCross)
│   │   └─ Formato Rojo (opcional)
│   └─ Paso 5: ¿Resalte instrucción? (Sí/No)
│       └─ Si sí: Seleccionar celda + Aplicar formato amarillo
```

```

├─ Paso 6: ¿Mensaje de alerta? (v2.2.2) (Sí/No)
│   └─ Si sí: Ingresar texto del mensaje
│       └─ Seleccionar ubicación
│           └─ Combinar celdas si necesario
│               └─ Aplicar formato rojo (Bold, Centered)
│                   └─ Inyectar fórmula: IF(COUNTIF>COUNTA)
└─ Mostrar modo aplicado: "Fila por Fila" o "Búsqueda Global"
else
    └─ Mostrar "No hay validación marcada"

```

### 5.3 Método: btnAplicar\_Click - Validación de Catálogos (Detalle)

#### **Novedad en v2.2.2:** Entrada manual vs seleccionar celdas

```

DialogResult tipoEntrada = MessageBox.Show(
    "¿Deseas escribir el valor de la lista manualmente o seleccionar celdas?",
    "Origen del Catálogo",
    MessageBoxButtons.YesNo,
    MessageBoxIcon.Question);

if (tipoEntrada == DialogResult.Yes)
{
    // === CASO 1: ENTRADA MANUAL ===
    object resultadoTexto = excelApp.InputBox("Escribe las opciones (ej:
1,2,9)...");
    string textoEscrito = resultadoTexto.ToString().Trim();
    formulaOpciones = textoEscrito.Replace(",", separador);
}
else
{
    // === CASO 2: SELECCIONAR CELDAS ===
    object resultadoInput = excelApp.InputBox("Selecciona el rango de
opciones...");
    Excel.Range rangoOrigen = (Excel.Range)resultadoInput;

    bool esCeldaUnica = (rangoOrigen.Count == 1) || (bool)rangoOrigen.MergeCells;

    if (esCeldaUnica)
    {
        // Preguntar: ¿Números? ¿Texto? ¿Referencia?
        DialogResult respuesta = MessageBox.Show(
            "¿Extraer SOLO NÚMEROS o mantener TEXTO EXACTO o usar como
REFERENCIA?",
            "Configuración de Catálogo",
            MessageBoxButtons.YesNoCancel);

        if (respuesta == DialogResult.Cancel)
        {

```

```

        // Usar como referencia
        formulaOpciones = "=" + rangoOrigen.Address;
    }
    else if (respuesta == DialogResult.Yes)
    {
        // Extraer solo números
        // ... lógica de limpieza
    }
    else
    {
        // Mantener texto exacto
        // ... lógica de limpieza sin números
    }
}
}

```

## 5.4 Método: btnAplicar\_Click - Validación de Bloqueo (Detalle Completo v2.2.2)

### Paso 1: Detección Automática de Tipo de Rango

```

bool esFilaPorFila = false;

if (rangoCondicion.Count == 1)
{
    esFilaPorFila = true;
}
else if (_rangoCapturado.Rows.Count == rangoCondicion.Rows.Count)
{
    DialogResult respFila = MessageBox.Show(
        "He detectado que el rango de condición tiene el MISMO número de  

        filas...\n\n" +
        "¿Deseas que la regla se aplique FILA POR FILA?",
        "Evaluación Inteligente de Matrices",
        MessageBoxButtons.YesNo,
        MessageBoxIcon.Question);

    esFilaPorFila = (respFila == DialogResult.Yes);
}

```

### Paso 6: Mensaje de Alerta Personalizado (Nuevo v2.2.2)

```

DialogResult respuestaAlerta = MessageBox.Show(
    "¿Deseas agregar un mensaje de alerta en alguna celda específica?",
    "5. Mensaje de Alerta Especial",
    MessageBoxButtons.YesNo,
    MessageBoxIcon.Question);

if (respuestaAlerta == DialogResult.Yes)

```

```

{
    // 1. Obtener texto del mensaje
    object resultadoTexto = excelApp.InputBox(
        "Escribe el texto del mensaje de alerta (Ej: 'Especifique el nombre'):",
        "Texto del Mensaje",
        Type.Missing, Type.Missing, Type.Missing, Type.Missing, Type.Missing, 2);

    string textoAlerta = resultadoTexto.ToString().Trim();

    // 2. Obtener ubicación
    object resultadoRangoAlerta = excelApp.InputBox(
        "Selecciona la celda o rango donde aparecerá este mensaje:",
        "Ubicación del Mensaje",
        Type.Missing, Type.Missing, Type.Missing, Type.Missing, Type.Missing, 8);

    Excel.Range rangoAlerta = (Excel.Range)resultadoRangoAlerta;

    // 3. Combinar celdas si es necesario
    if (rangoAlerta.Count > 1)
    {
        rangoAlerta.Merge();
    }

    // 4. Aplicar formato rojo (Bold, Centered)
    rangoAlerta.Font.Bold = true;
    rangoAlerta.Font.Color =
    System.Drawing.ColorTranslator.ToOle(System.Drawing.Color.Red);
    rangoAlerta.HorizontalAlignment = Excel.XlHAlign.xlHAlignCenter;
    rangoAlerta.VerticalAlignment = Excel.XlVAlign.xlVAlignCenter;

    // 5. Construir fórmula en INGLÉS UNIVERSAL
    string dirCapturadaGlobal = _rangoCapturado.Address;
    string formulaAlerta = $"=IF(COUNTIF({dirCondicionGlobal},
    {criterioContarSi})>COUNTA({dirCapturadaGlobal}),\"{textoAlerta}\",\"\")";

    // 6. Inyectar fórmula usando .Formula (no .FormulaLocal)
    rangoAlerta.Formula = formulaAlerta;
}

```

## 5.5 Métodos: CheckedChanged

**Propósito:** Validar precondiciones cuando el usuario marca/desmarca checkboxes

```

private void chkCatalogos_CheckedChanged(object sender, EventArgs e)
{
    if (chkCatalogos.Checked)
    {
        if (_libroCenso == null || _rangoCapturado == null)
        {
            MessageBox.Show("Primero carga un censo y captura un rango.",

```

```

"Aviso");
        chkCatalogos.Checked = false;
    }
}

private void chkNS_CheckedChanged(object sender, EventArgs e)
{
    if (chkNS.Checked)
    {
        if (_libroCenso == null || _rangoCapturado == null)
        {
            MessageBox.Show("Primero carga un censo y captura un rango.",
"Aviso");
            chkNS.Checked = false;
        }
    }
}

```

## 5.6 Método: ObtenerNumeroPregunta

**Responsabilidad:** Detectar automáticamente el identificador de la pregunta

```

private string ObtenerNumeroPregunta(Excel.Range rango)
{
    try
    {
        Excel.Worksheet hoja = rango.Worksheet;
        int filaInicial = rango.Row;

        // Buscar hacia arriba en la columna A
        for (int f = filaInicial; f >= 1; f--)
        {
            Excel.Range celdaA = hoja.Cells[f, 1];
            object valor = celdaA.Value2;

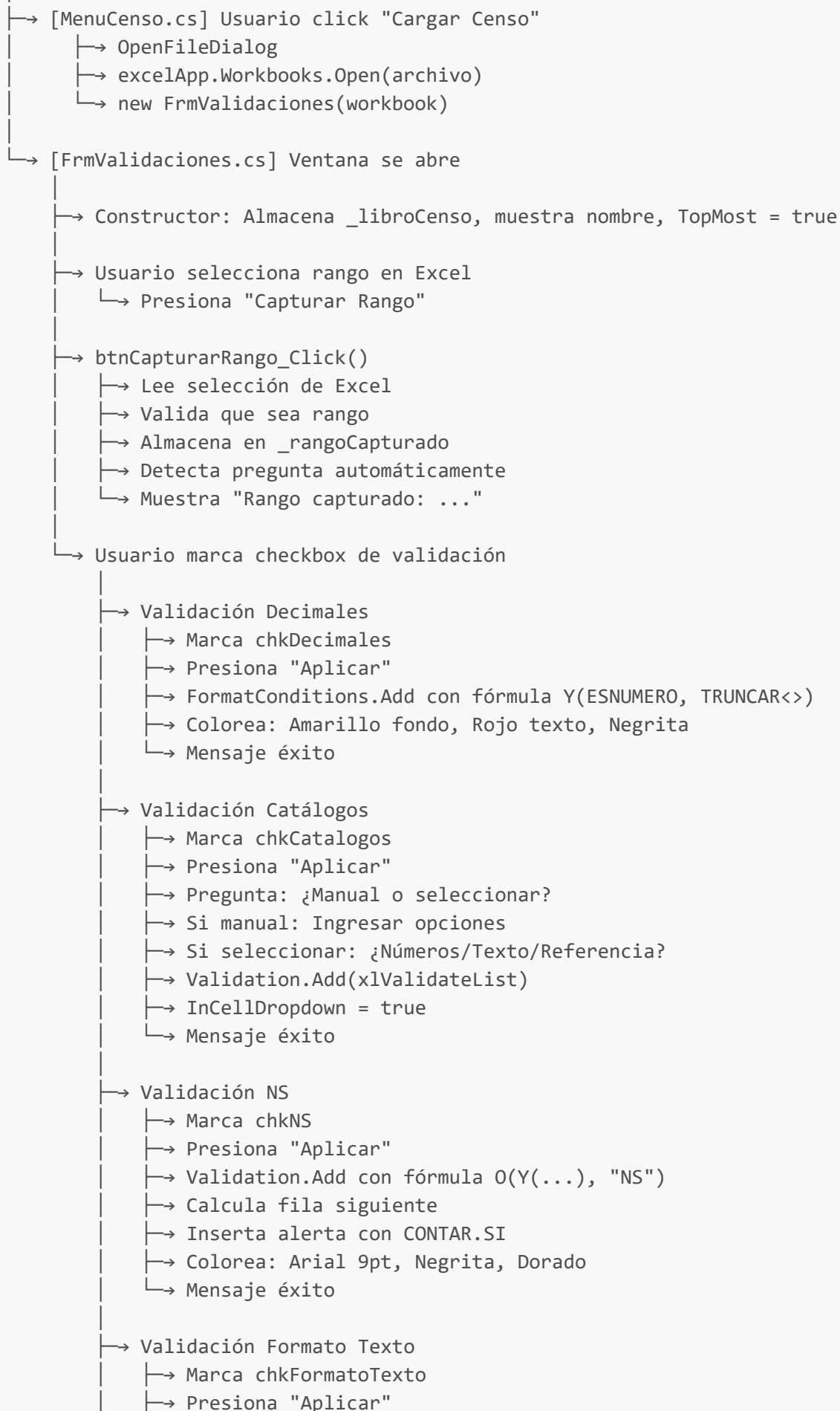
            if (valor != null && !string.IsNullOrEmpty(valor.ToString().Trim()))
            {
                return valor.ToString().Trim();
            }
        }
    }
    catch { /* Silencio */ }

    return "(no encontrada)";
}

```

## Flujo de Trabajo Completo

## INICIO



```

└─ Validation.Add con fórmula Y(IGUAL, ESPACIOS)
└─ Configura mensaje: "Mayúsculas, sin espacios..."
└─ Mensaje éxito

└─ Validación Bloqueo Dinámico (Complejo - v2.2.2)
    └─ Paso 1: Seleccionar condición
        └─ Detecta: 1 celda / igual tamaño / diferente
            └─ Decide automáticamente: esFilaPorFila

    └─ Paso 2: Seleccionar operador (=, <>, >, <, >=, <=)

    └─ Paso 3: Ingresar valor

    └─ Paso 4: ¿Resalte rojo? (Sí/No)
        └─ Si Sí: Formato rojo cuando desbloqueado + vacío

    └─ Aplicar Data Validation + Formatos
        └─ Data Validation: CONTAR.SI (fila-por-fila)
        └─ Formato Gris: CONTAR.SI=0
        └─ Formato Rojo: Y(CONTAR.SI>0, ESBLANCO)

    └─ Paso 5: ¿Resalte instrucción? (Sí/No)
        └─ Si Sí: Seleccionar celda con instrucción
        └─ Aplicar formato amarillo
        └─ Usa dirCondicionGlobal en fórmula

    └─ Paso 6: ¿Mensaje de alerta? (v2.2.2) (Sí/No)
        └─ Si Sí: Ingresar texto personalizado
        └─ Seleccionar ubicación
        └─ Combinar celdas si es necesario
        └─ Aplicar formato: Rojo Bold Centered
        └─ Inyectar fórmula: IF(COUNTIF>COUNTA)
        └─ Fórmula usa .Formula (INGLÉS universal)

    └─ Mensaje final: "Bloqueo aplicado. Modo: Fila por Fila / Búsqueda
Global"

```

## Especificaciones Técnicas

### Requisitos del Sistema

| Aspecto                   | Requisito                        |
|---------------------------|----------------------------------|
| <b>Versión .NET</b>       | .NET Framework 4.8               |
| <b>Versión C#</b>         | 7.3                              |
| <b>Versión Excel</b>      | 2013 o superior                  |
| <b>SO</b>                 | Windows 7/8/10/11 (32 o 64 bits) |
| <b>Memoria RAM Mínima</b> | 4 GB                             |

| Aspecto          | Requisito                            |
|------------------|--------------------------------------|
| Espacio en Disco | ~100 MB                              |
| Bitness          | Mismo que Office instalado (32 o 64) |

Dependencias Principales

```
Microsoft.Office.Interop.Excel (versión 15.0+)
ExcelDna.Integration (versión 0.34+)
System.Windows.Forms
System.Drawing
System.Collections.Generic
```

Internacionalización

Funciones Localizadas:

| Español:    | Inglés:    | Función                 |
|-------------|------------|-------------------------|
| ESNUMERO()  | ISNUMBER() | Verificar si es número  |
| TRUNCAR()   | TRUNC()    | Truncar decimales       |
| CONTAR.SI() | COUNTIF()  | Contar con condición    |
| SI()        | IF()       | Condicional             |
| Y()         | AND()      | Y lógico                |
| O()         | OR()       | O lógico                |
| MAYUSC()    | UPPER()    | Convertir a mayúsculas  |
| ESPACIOS()  | TRIM()     | Remover espacios extra  |
| IGUAL()     | EXACT()    | Comparación exacta      |
| LARGO()     | LEN()      | Largo de texto          |
| ESBLANCO()  | ISBLANK()  | Verificar si está vacío |

Separador de Argumentos (Dinámico):

```
string separador = excelApp.International[
    Excel.XlApplicationInternational.xlListSeparator].ToString();
// España: ";" | USA: "," | Francia: ";"
```

Rendimiento

| Operación            | Tiempo Estimado |
|----------------------|-----------------|
| Apertura formulario  | < 500 ms        |
| Captura de rango     | Inmediato       |
| Validación decimales | 1-2 seg         |



| Operación                             | Tiempo Estimado |
|---------------------------------------|-----------------|
| Validación catálogos                  | 2-3 seg         |
| Validación NS                         | 3-5 seg         |
| Validación formato texto              | 1-2 seg         |
| Validación bloqueo completa (6 pasos) | 15-25 seg       |
| Memoria usada                         | 50-100 MB       |

## Límites Conocidos

- **Máximo rango:** 1,048,576 filas × 16,384 columnas
- **Máximo opciones catálogo:** ~256 opciones
- **Máximo formato condicional:** 3 reglas por celda
- **Máximo texto de mensaje:** 255 caracteres
- **Máximo criterio CONTAR.SI:** ~256 caracteres

## Patrones de Código v2.2.2

### Pattern 1: Validación de Selección

```
if (seleccion is Excel.Range)
{
    _rangoCapturado = (Excel.Range)seleccion;
    // Proceder
}
else
{
    MessageBox.Show("Por favor, selecciona celdas de Excel.");
}
```

### Pattern 2: Construcción de Fórmulas Dinámicas

```
string separador = excelApp.International[
    Excel.XlApplicationInternational.xlListSeparator].ToString();

string formula = $"=Y(ESNUMERO({direccion}){separador}TRUNCAR({direccion})<>
{direccion})";
```

### Pattern 3: Detección Automática de Matrices

```
bool esFilaPorFila = false;

if (rangoCondicion.Count == 1)
```

```
{
    esFilaPorFila = true;
}
else if (_rangoCapturado.Rows.Count == rangoCondicion.Rows.Count)
{
    DialogResult resp = MessageBox.Show("¿Fila por Fila?", ...);
    esFilaPorFila = (resp == DialogResult.Yes);
}
```

## Pattern 4: Fórmula Universal para Compatibilidad

```
// Usar .Formula (INGLÉS) para máxima compatibilidad multiidioma
string formulaAlerta = $"=IF(COUNTIF({dirGlobal},
{criterio})>COUNTA({dirCapturada}),\"{texto}\",\"\\\")";
rangoAlerta.Formula = formulaAlerta; // NO .FormulaLocal
```

## Troubleshooting v2.2.2

### Problema 1: Data Validation no restringe entrada

**Causa:** Tipo de validación incorrecto

**Solución:**

```
// Para fórmula personalizada
Excel.XlDVType.xlValidateCustom

// Para lista de opciones
Excel.XlDVType.xlValidateList
```

### Problema 2: Bloqueo no funciona en matriz

**Causa:** Tamaños de rango no coinciden correctamente

**Diagnóstico:**

```
int filasCaptura = _rangoCapturado.Rows.Count;
int filasCondicion = rangoCondicion.Rows.Count;
System.Diagnostics.Debug.WriteLine($"Captura: {filasCaptura}, Condición:
{filasCondicion}");
```

### Problema 3: Mensaje de alerta no aparece

**Causa:** La fórmula usa .FormulaLocal en lugar de .Formula

**Solución:**

```
// CORRECTO
rangoAlerta.Formula = "=IF(COUNTIF(...),\"mensaje\",\"\")";

// INCORRECTO
// rangoAlerta.FormulaLocal = "=SI(CONTAR.SI(...),\"mensaje\",\"\")";
```

**Problema 4:** Separador incorrecto en fórmulas

**Causa:** El sistema usa separador local pero la fórmula es en INGLÉS

**Solución:**

```
// Para .Formula (INGLÉS), usar coma
string formula = "=IF(A1=1,\"SÍ\",\"\")";
rangoAlerta.Formula = formula;

// Para .FormulaLocal (ESPAÑOL), usar punto y coma
string formulaLocal = "=SI(A1=1;\"SÍ\";\"\")";
rangoAlerta.FormulaLocal = formulaLocal;
```

## Resumen de Cambios v2.2.2

### 🌟 Nuevas Características

#### 1. Entrada Manual en Catálogos (v2.2.2)

- Usuario puede escribir opciones directamente
- Alternativa a seleccionar celdas
- Ideal para listas pequeñas (1,2,9)

#### 2. Mensaje de Alerta Personalizado en Bloqueo (v2.2.2)

- Paso 6 adicional en flujo de bloqueo
- Usuario ingresa texto personalizado
- Se muestra automáticamente cuando se cumple condición + vacío
- Fórmula universal en INGLÉS para máxima compatibilidad

#### 3. Mejoras en Catálogos

- Opción YesNoCancel para 3 modos: Números, Texto, Referencia
- Mejor feedback visual sobre qué se está extrayendo
- Compatible con celdas únicas y rangos múltiples

### 🔧 Mejoras Técnicas

- ✓ Fórmulas universales usando sintaxis INGLÉS
- ✓ Mejor separación de responsabilidades (Local vs Global)
- ✓ Compatibilidad multiidioma mejorada
- ✓ Detección automática de matrices más inteligente

## Matriz de Validaciones Disponibles (v2.2.2)

| Validación       | Tipo                     | Motor              | Restricción | Versión  |
|------------------|--------------------------|--------------------|-------------|----------|
| Decimales        | FormatCondition          | ESNUMERO + TRUNCAR | No          | v1.0     |
| Catálogos        | Data Validation (List)   | Rango/Manual       | Sí          | v2.2.2 ★ |
| NS               | Data Validation (Custom) | O(Y(...), "NS")    | Sí          | v2.0     |
| Formato Texto    | Data Validation (Custom) | IGUAL + ESPACIOS   | Sí          | v2.1     |
| Bloqueo Dinámico | Data Validation + Format | CONTAR.SI          | Sí          | v2.2.2 ★ |

## Conclusión

SAVCNG ExcelDNA en versión 2.2.2 es un complemento robusto y versátil con cinco tipos de validación complementarios. Las mejoras en catálogos y el mensaje de alerta personalizado en bloqueo representan saltos cualitativos hacia una herramienta más potente y flexible.

**Fortalezas de v2.2.2:** ✓ Interfaz intuitiva y guiada ✓ Soporte multiidioma automático ✓ Validaciones versátiles y complementarias ✓ Detección automática de contexto ✓ Restricción de entrada en tiempo real ✓ Lógica condicional compleja pero accesible ✓ Soporte para matrices y datos paralelos ✓ Mensajes personalizados y dinámicos ✓ Entrada manual en catálogos (nueva)

### Recomendaciones para v3.0:

- Historial de auditoría de cambios
- Validaciones de rango (mín/máx)
- Validaciones de fecha
- Exportación/importación de reglas (JSON)
- Caché de configuraciones frecuentes
- Duplicación de validaciones entre rangos
- Vista previa de validaciones

**Versión:** 2.2.2.0

**Última Actualización:** 12 de junio de 2026

**Estado:** En Desarrollo Continuo

**Rama de Git:** Desarrollo

## Apéndice A: Cambios desde v2.2.1 a v2.2.2

Archivo: FrmValidaciones.cs

**Adiciones:**

- Flujo de entrada manual en validación de catálogos (líneas ~175-195)
- Paso 6 de mensaje de alerta personalizado en bloqueo (líneas ~621-672)
- Opción YesNoCancel para seleccionar entre 3 modos en catálogos

#### Cambios:

- Mejora en pregunta de catálogos: "¿Manual o seleccionar?"
- Fórmula universal en .Formula para máxima compatibilidad
- Mejor manejo de dirección global en mensajes de alerta

**Líneas Modificadas:** ~95 líneas **Líneas Agregadas:** ~55 líneas (Paso 6 de bloqueo + entrada manual)

## Apéndice B: Equivalencias de Funciones Excel

### Funciones de Validación

| Operación           | Español     | Inglés     | Uso                    |
|---------------------|-------------|------------|------------------------|
| Verificar número    | ESNUMERO()  | ISNUMBER() | Decimales, NS          |
| Truncar             | TRUNCAR()   | TRUNC()    | Decimales              |
| Igualdad exacta     | IGUAL()     | EXACT()    | Formato Texto          |
| Mayúsculas          | MAYUSC()    | UPPER()    | Formato Texto          |
| Trimmed             | ESPACIOS()  | TRIM()     | Formato Texto          |
| Contar con criterio | CONTAR.SI() | COUNTIF()  | Bloqueo, NS            |
| Condicional         | SI()        | IF()       | Alertas                |
| Y lógico            | Y()         | AND()      | Formato Texto, Bloqueo |
| O lógico            | O()         | OR()       | NS                     |
| Está vacío          | ESBLANCO()  | ISBLANK()  | Bloqueo (Rojo)         |